

Review of Relational Databases

Overview

- **Describe the relational database model**
- **Explain basic SQL functions**
- **Recognize the OMOP database**

Async

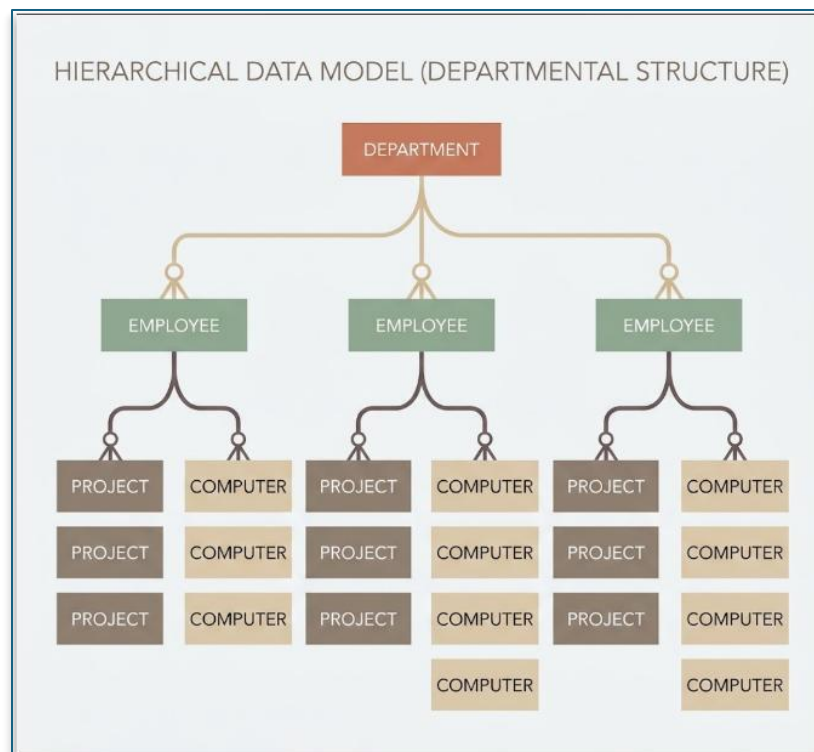
The Relational Model

Data Models

- a collection of concepts used to describe:
 - structure of a database
 - relationships between data elements
- the history of databases is closely tied to the development of different data models
 - conceptual data model
 - high-level
 - describes how users understand relationships between data
 - main focus of this course
 - physical data model
 - more technical
 - describes how data is physically stored on computer hardware
 - *not a major focus of this course*

Hierarchical Model – 1960s

- **organizes** data in a **top-down** tree structure
- primarily supports **one-to-many relationships**
 - a department
 - can have many employees
 - an employee
 - belongs to only one department
 - can have many projects
 - can have many computers
 - a project
 - belongs to only one employee
 - a computer
 - belongs to only one person



Gemini 3.7 Flash

- **key rule**
 - every child can only have one parent
- **query limitation**
 - must begin at the top of the hierarchy

Network Model – 1960s	• the network model expanded the hierarchical model • its major improvement was support for many-to-many relationships • advantages ○ more flexible than the hierarchical model • problems ○ difficult to maintain ○ structural changes can be complicated ○ not every real-world relationship fits naturally into a parent-child structure • a more flexible model is still needed
Relational Model – 1970s	• the relational model was developed primarily through the work of E. F. Codd • it became the foundation of modern relational databases • a major improvement was separating physical data storage from conceptual presentation of data • this makes databases easier to: ○ query ○ update ○ manage
Properties of Relational Databases	• data is stored in tables of rows and columns • primary keys (or <i>unique key</i>) ○ a unique identifier for a row in a table • relations have no inherent order • columns have no inherent order • attributes must be atomic (can only contain one value)

	• NULL values ○ literally “nothing” – not “zero” ○ $\text{NULL} \neq \text{NULL}$
Relational Model: Constraints	
Constraints	• a rule used when designing and working with a relational database • help control: ○ what data can be stored ○ which values are valid ○ how records are uniquely identified ○ how different tables may reference one another
Three Types of Constraints	• implicit ○ built into the relational database system ○ the user does not normally choose or manage these • explicit ○ rules deliberately defined by the database designer • business rules ○ usually more complex rules associated with the application using the database ○ instead of allowing invalid data to reach the database, the application layer (ex: a web form) can enforce these rules first
Domain Constraints	• defines what kind of data can appear in an attribute or column • commonly enforced using data types that are normally defined when the table is created • values stored in that column must follow the defined domain

Key Constraints	• primary key uniquely identifies one row in a relation • each value must be unique
Key Design Must Account for Future Data	• a key may work with the database's current data but fail when new records are added late • therefore, database designers should consider ○ existing records ○ expected future records ○ whether the chosen key will remain unique • a poor key design can prevent legitimate future data from being stored
Composite Keys	• uses multiple attributes together to uniquely identify a row
Database Schema	• a map of the database • every relation has a schema, and the schemas of all relations together describe the overall database structure • looking at the complete schema helps designers make decisions based on the entire database, rather than designing each table independently
Entity Integrity	• an entity integrity constraint is an implicit database constraint ○ example: a primary key can never be null
Referential Integrity	• referential integrity controls relationships between different relations ○ example: if a record in Table B refers to a record in Table A, the referenced record must actually exist in Table A

Foreign Keys

- a foreign key
 - is an attribute that references the primary key of another relation
 - does not have to be the primary key of its own table
 - only needs to reference the appropriate primary key in another table
- foreign keys are essential because they allow relational databases to:
 - connect tables
 - query information across tables
 - represent relationships between entities

SQL Basics**FROM, SELECT, and WHERE**

- SQL queries are built from three fundamental clauses
 - required clauses:
 - FROM
 - which table(s) provide the data
 - SELECT
 - which columns appear in the output
 - optional clause:
 - WHERE
 - which rows are returned

```
SELECT what you want to see
FROM   where the data comes from
WHERE  which records you want
```

FROM: Choose the Table

- tells the database which table contains the data
- table aliases
 - a table can be given a shorter temporary name called an alias

```
FROM table_patient p
```

- now '*p*' represents '*table_patient*' for the rest of the query

SELECT: Choose the Columns

- determines which columns appear in the query output

```
SELECT
    p.patient_name,
    p.patient_dob,
    p.patient_race
FROM table_patient p;
```

- the table may contain many more columns, but only these three will appear

- qualifying column names
 - with its table name

```
table_patient.patient_name
```

- with the table's alias

```
p.patient_name
```

SQL Functions in SELECT	• transform or calculate values before they appear in the output <pre>SELECT UPPER(p.patient_name) FROM table_patient p;</pre> ○ UPPER() converts to uppercase ○ so 'Joe Waller' becomes 'JOE WALLER'
Renaming Output Columns	• a calculated or transformed column can be given an alias using AS <pre>SELECT UPPER(p.patient_name) AS upper_name FROM table_patient p;</pre> ○ does not rename the column in the original database ○ only changes the name shown in the query output
Combining Column Values	• SQL functions can also combine values from multiple columns <pre>SELECT CONCAT(first_name, ' ', last_name) AS full_name</pre>
Core Functions	String Functions • UPPER() ○ converts text to uppercase • LOWER() ○ converts text to lowercase • concatenation ○ combines two or more string values into a single value Aggregate Functions • COUNT() ○ counts the number of returned records or values

	• AVG() ○ calculates the average of numeric values Mathematical Functions • FLOOR() ○ rounds a number down • CEILING() ○ rounds a number up • Standard mathematical functions ○ perform common arithmetic calculations Date Functions • date-difference functions ○ calculate the amount of time between dates • age/date calculations ○ calculate values such as a person's age from stored dates • Notes ○ functions can be used in both SELECT and WHERE clauses ○ exact function syntax can vary between relational database management systems ○ this course uses PostgreSQL (Postgres) syntax

	•
	•
	•
	•

